Automatic Piecewise Linear Regression version 10.24.0

Von Ottenbreit Data Science

Automatic Piecewise Linear Regression (APLR)

- Automatically handles variable selection, non-linear relationships and interactions.
- Empirical tests show that APLR is often able to compete with tree-based methods on predictiveness.
- APLR produces interpretable models.
- APLR produces smoother predictions than tree-based methods.
- APLR can be used for regression and classification tasks, including multiclass classification.

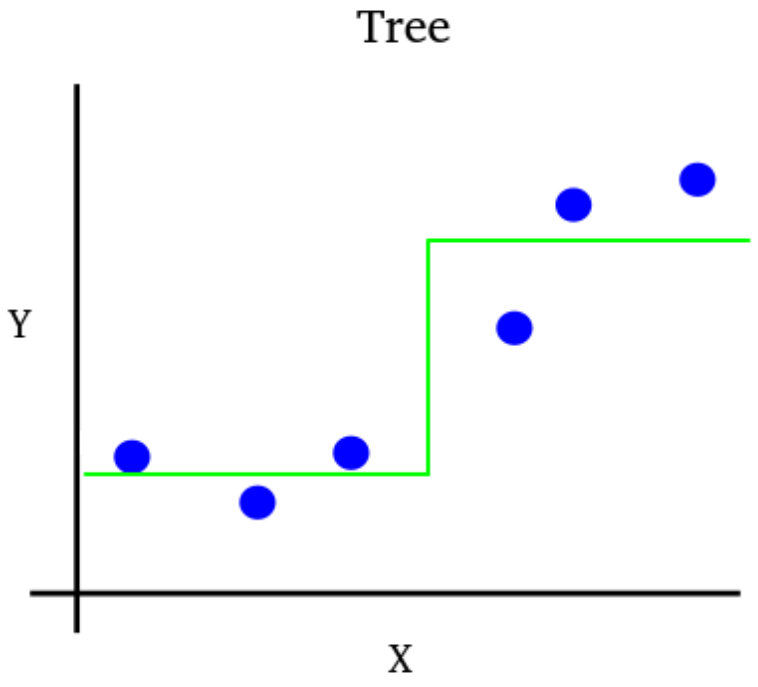
Some useful functionality

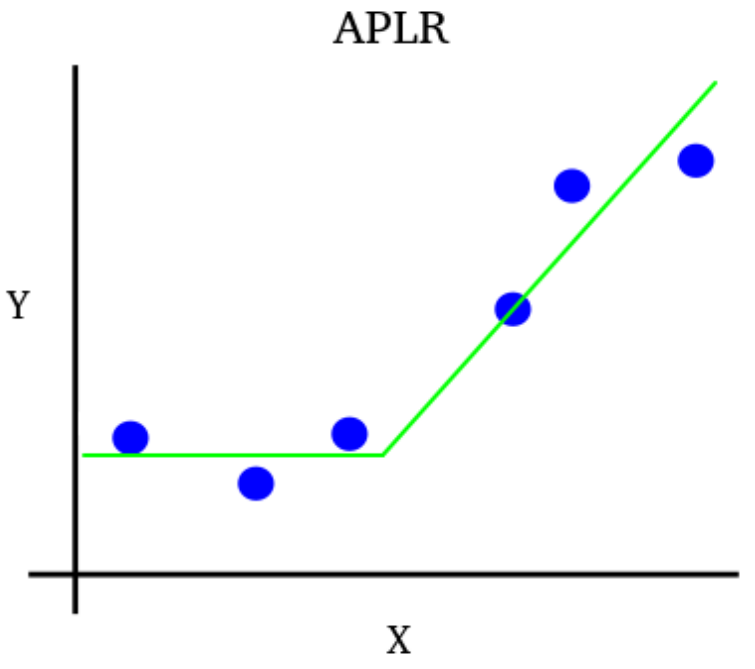
- Allows to specify the loss function among several built-in options such as *mse* (default), *poisson*, *gamma*, etc.
- Allows to specify the link function among built in variants: *identity* (default), *logit* and *log*.
- Allows to specify the function for calculating the validation set tuning metric among built-in variants such as *default* (same as loss function), *mse*, *negative_gini,* etc.
- Alternatively allows to pass custom Python functions for each of the above.
- The user can specify a list of predictors with monotonic constraints. This can improve model realism, usually with only a minimal loss increase.
- The user can provide constraints on which predictors are allowed in interaction terms.
- It is possible to get the shape of each main effect and interaction. This makes it easier to interpret the model, for example by plotting the shapes of main effects or two-way interactions.
- It is possible to calculate local (observation specific) feature importance as well as local contributions to the linear predictor from each feature in the model.
- Automatically handles missing predictor values and categorical predictors.
- Regarding classification, the object APLRClassifier fits a logit APLR model for each response category. When predicting, each observation is predicted to the class with the highest predicted class probability.

Tuning APLR

- APLR is often robust with the default settings. However, it is usually possible to get better results by tuning. Below are the most important tuning parameters.
- *m* (default is 3000) is the maximum boosting steps to try. Should be large enough to minimize the validation error. The default value is not always enough. Early stopping often prevents unnecessary computational costs associated with a high *m*.
- *v* (default is 0.5) is the learning rate. Must be greater than zero and not more than one. The higher the faster the algorithm learns and the lower *m* is required, reducing computational costs potentially at the expense of predictiveness. Empirical evidence suggests that *v* <= *0.5* gives good results for APLR. For datasets with weak signals or small sizes, a low learning rate, such as 0.1, may be beneficial.
- *max_interaction_level* (default is 1) specifies the maximum allowed interaction depth. Tune, for example by doing a grid search, for best predictiveness. For best interpretability use 0 (or 1 if interactions are needed).
- *min_observations_in_split* (default is 0.3) specifies the minimum number of observations that a term in the model must rely on as well as the minimum number of boundary value observations where there cannot be splits. Values below 1.0 are interpreted as a power of the total number of training observations. For example, with 10,000 training observations, a value of 0.5 results in a minimum of ceil(10000**0.5) = 100 observations. This hyperparameter should be tuned. Larger values are more appropriate for larger datasets. Larger values result in more robust models (lower variance), potentially at the expense of increased bias.
- *ridge_penalty* (default is 0.0001) specifies the (weighted) ridge penalty applied to the model. Positive values can smooth model effects and help mitigate boundary problems, such as regression coefficients with excessively high magnitudes near the boundaries. To find the optimal value, consider using a grid search or similar. Negative values are treated as zero.
- *penalty_for_non_linearity* (default is 0.0) and *penalty_for_interactions* (default is 0.0) set penalties on non-linear and interaction terms, respectively (maximum penalty is 1.0). Increasing these values will increase interpretability but may decrease predictiveness.

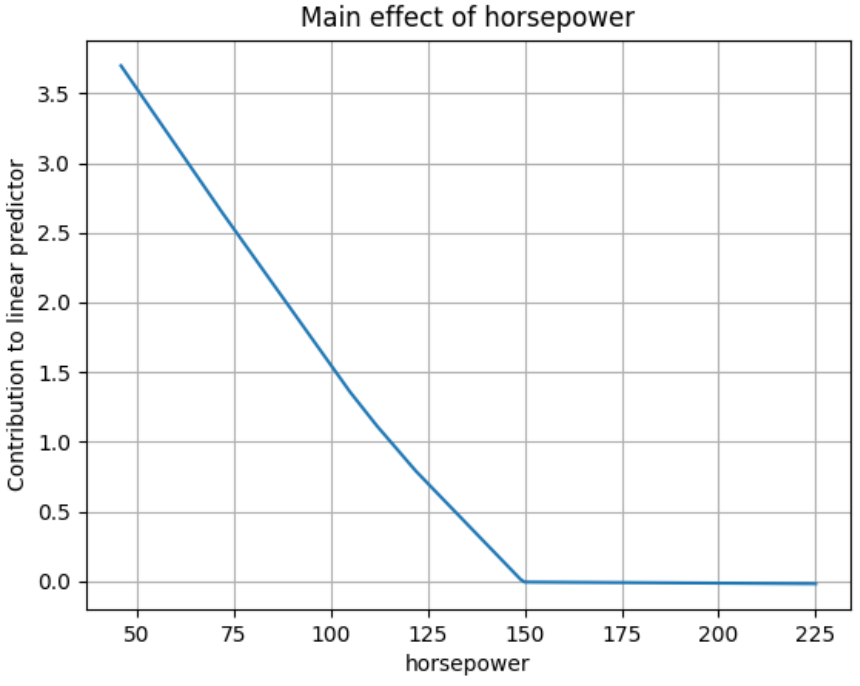
Some differences compared to tree-based methods

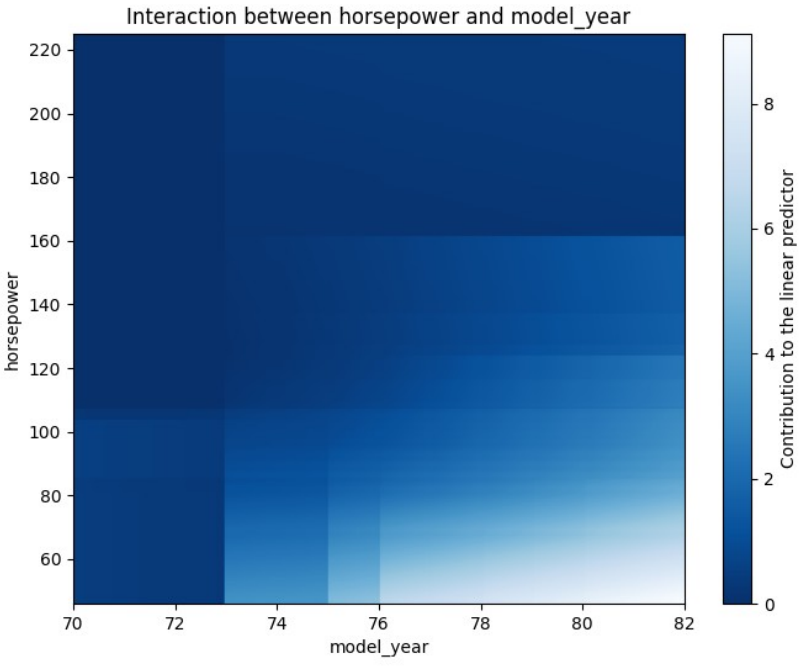




- Trees fit piecewise constants.
- APLR fits piecewise linear basis functions or linear effects.
- Trees often only fit interactions (unless for example max tree depth is 1).
- APLR fits main effects and potentially interactions (often simpler to interpret).

Interpretation example on the Auto MPG dataset





- **The APLR model in this example predicts miles per gallon (*mpg*) for cars and was trained on the Auto MPG dataset. The model was allowed to use two-way interactions (*max_interaction_level* = 1).**
- **The above charts were generated by using the *plot_affiliation_shape* method in APLR.**
- **The left chart shows the combined effect of all main (non-interaction) terms involving *horsepower*. As *horsepower* increases, predicted *mpg* decreases, but the decline levels off once *horsepower* exceeds about 150.**
- **The right chart shows the combined effect of all interaction terms between *horsepower* and *model_year*. An increase in *model_year* increases predicted *mpg* and the effect is greatest when *horsepower* is low.**

References

- **Link to published article:** https://link.springer.com/article/10.1007/s00180-024-01475-4